

1. Pipeline de traitement avec ImageJ

Un étudiant a utilisé ImageJ pour compter le nombre d'arbres dans l'image aérienne de la Fig.1. Le script en pseudo-code ci-dessous contient l'historique de toutes les commandes et essais réalisés par l'étudiant.

Fig.1 : Vue aérienne d'une plantation d'oliviers. ►

Fig.2 : Une image 8-bit. ▼

1 3 3 5 2
4 1 5 7 8
6 6 9 2 3



Q1 : Quelle est la largeur et la hauteur de l'image de la Fig.1 ? Quel est le type de l'image ? Justifiez votre réponse.

Q2 : Comment fait-on en Python pour lire la largeur, la hauteur et le type d'une image chargée avec scikit-image ?

Q3 : Décrire le principe de fonctionnement de la commande `run("Median...", "radius=1")`. L'appliquez à l'image de la Fig. 2 pour le pixel de coordonnées [2,1].

Q4 : Appliquez `run("Mean...", "radius=1")` à l'image de la Fig. 2 pour le pixel de coordonnées [2,1].

Q5 : Commentez le script ci-joint. Indiquez combien de pipelines (ou essais) ont été testés puis pour chacun des pipelines, décrire la suite d'opérations (si nécessaire, supprimer les lignes de pseudo-code qui ne servent à rien) ainsi que l'approche de traitement d'images utilisés. Quel est le résultat de ces pipelines ? Soyez le plus précis possible.

Q6 : Proposez un pipeline de traitement de l'image de la Fig.1 différent de ceux testés dans le script.

```
open("data/OlivoTotal.png")
selectImage("OlivoTotal.png")
run("Split Channels")
selectImage("OlivoTotal.png (red)")
selectImage("OlivoTotal.png (green)")
selectImage("OlivoTotal.png (blue)")
selectImage("OlivoTotal.png (red)")
run("Median...", "radius=4")
run("Invert")
run("Subtract Background...", "rolling=50")
run("Enhance Contrast", "saturated=0.35 normalize")
run("Find Maxima", "prominence=20 output=[Selection]")
run("Enlarge", "enlarge=6")
newImage("results", "8-bit black", 643, 605, 1)
selectImage("OlivoTotal.png (red)")
selectImage("results")
run("Restore Selection")
run("Draw", "slice")
saveAs("PNG", "data/results.png")
open("data/OlivoTotal.png")
selectImage("OlivoTotal.png")
run("8-bit")
run("FFT")
selectImage("FFT of OlivoTotal.png")
run("Select All")
run("Find Maxima", "prominence=20 output=[Selection]")
run("Enlarge...", "enlarge=6")
setForegroundColor(255, 255, 255)
setBackgroundColor(0, 0, 0)
run("Fill", "slice")
run("Inverse FFT")
saveAs("PNG", "data/Inverse FFT of OlivoTotal.png")
selectImage("FFT of OlivoTotal-1.png")
saveAs("PNG", "data/FFT of OlivoTotal-1.png")
selectImage("Inverse FFT of OlivoTotal.png")
run("Invert")
run("Find Maxima", "prominence=20 output=[Selection]")
newImage("resultsfft", "8-bit black", 643, 605, 1)
selectImage("Inverse FFT of OlivoTotal.png")
selectImage("resultsfft")
run("Restore Selection")
run("Enlarge", "enlarge=6")
run("Draw", "slice")
saveAs("PNG", "data/resultsfft.png")
selectImage("Inverse FFT of OlivoTotal.png")
run("Enhance Contrast", "saturated=0.35 normalize")
selectImage("resultsfft.png")
```

2. Implantation du script ImageJ en Python

Q7 : On souhaite faire fonctionner ce script sous Python. Combien de fonctions faut-il implanter ? Lister toutes les fonctions à implanter et pour chacune d'elles, indiquez leur signature.

Note : La **signature** est l'ensemble des paramètres (avec leurs noms, positions, valeurs par défaut et types), ainsi que le type de retour d'une fonction. Par exemple, la fonction Python « puissance » `pow(base, exp, mod=None)` qui calcule $base^{exp} \% mod$ a pour signature `pow(base:Number, exp:Number, mod=None:Number) → Number`. Si la fonction ne retourne rien, on indique `None`.

Q8 : Implanter la fonction `open( . . )` – sachant que cette commande affiche aussi l'image – en utilisant numpy, matplotlib et scikit-image.

Q9 : Implanter la fonction `newImage(..)` en utilisant numpy, matplotlib et scikit-image.

Q10 : Dans ImageJ, quel est le rôle de la fonction `selectImage(..)`. Proposez une implantation en utilisant numpy et scikit-image.

Remarque : Les questions **Q5** et **Q6** sont les plus importantes de ce DST et donc auront un coefficient plus élevé que les autres questions.